

Related Paper Review 1

Human-In-the-Loop Software Development Agents (HULA)

Project	A.R.T.E.M.I.S (F26-008), FAST NUCES Lahore
Team	Hashir Rauf (23L-2572), Hira Khalid (23L-2594), Syeda Doureesha Batool (23L-2651)
Supervisor	Mr. Mubasher Hussain

1 Citation and Publication Status

W. Takerngsaksiri, J. Pasuksmit, P. Thongtanunam, C. Tantithamthavorn, R. Zhang, F. Jiang, J. Li, E. Cook, K. Chen, and M. Wu, “Human-In-the-Loop Software Development Agents,” in *Proceedings of the 2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP 2025)*, Ottawa, Canada, pp. 342–352, IEEE, 2025. DOI: 10.1109/ICSE-SEIP66354.2025.00036

This is a peer-reviewed paper published in the proceedings of ICSE, the leading conference in software engineering. It is not a preprint. The work was carried out and deployed at Atlassian, so the results come from an industrial deployment rather than a laboratory study.

2 What the Paper Does

The paper presents HULA, a framework in which a large language model agent writes software under human supervision rather than autonomously. The authors observe that fully autonomous coding agents produce changes that engineers will not merge, because the engineer has no opportunity to correct the agent’s intent before code is written.

HULA therefore splits the work into three stages with a person in control at each boundary. An AI planner agent reads the work item and the repository and produces a coding plan, naming the files it intends to change. The engineer reviews that plan and may edit or reject it before anything is generated. An AI coding agent then implements only the approved plan, and the engineer reviews the resulting code before it becomes a pull request.

The framework was deployed inside Atlassian and integrated with Jira, so that work items could be handed to the agent directly. The evaluation covers both offline benchmarks and real usage by Atlassian engineers.

3 Results Reported

- Engineers approved the agent’s coding plan in approximately 82 percent of cases, indicating that a plan is a reviewable artefact that people can meaningfully judge before code exists.
- A substantially smaller proportion of generated pull requests were merged than plans approved, which shows that approval at the plan stage does not by itself guarantee acceptable code.
- Engineers reported that the framework reduced the effort of starting a task, particularly in forming an initial plan and writing code for straightforward work items.

- Output quality depended heavily on the quality of the context supplied to the agent, with insufficient context being a recurring cause of unusable output.

4 Strengths

The central strength is that the human-approval loop is validated in a real organisation rather than argued for in principle. The 82 percent plan approval rate is evidence that asking a person to approve a plan is a workable interaction, not an obstacle that users route around.

The separation of planning from generation is the second strength. By making the plan an explicit artefact that exists before any code is written, the framework gives the user a cheap point of intervention. Correcting a plan costs far less than reviewing and repairing generated code.

The deployment context is itself a contribution. Because the system ran inside a working engineering organisation and against real Jira work items, the reported friction is real friction rather than anticipated friction.

5 Weaknesses and Limitations

The scope is confined to software engineering tasks within a repository, and within an organisation whose engineers are expert reviewers. The paper does not address a non-expert user, who may be unable to judge whether a plan is correct.

The gap between plan approval and pull request merge is acknowledged but not resolved. A user may approve a reasonable plan and still receive code that cannot be used, which means approval at one stage does not transfer to confidence at the next.

The framework also has no reversal mechanism. Once a change has been applied, the paper offers no facility to undo it as a unit, relying instead on the surrounding version control system.

Finally, the agent operates with access to the repository as a whole. There is no notion of a user-granted boundary restricting which files the agent may read or modify.

6 Relationship to A.R.T.E.M.I.S

This paper is the closest published work to the interaction model used in our project, and it supports a design decision we had already made. Our system also produces an inspectable plan before acting, and holds that plan for the user to approve, edit or reject. The 82 percent approval figure is external evidence that this model is practical, which strengthens the case for our approval gate beyond our own questionnaire finding.

Our project differs in four respects. First, our system applies the loop to general desktop knowledge work rather than only to code, so the user is not assumed to be a professional engineer. Second, our code assistance never writes to a file at all: it emits a difference for the user to apply, whereas HULA generates code into a branch. Third, we add reversal, so an approved plan can be undone as the single unit that was approved. Fourth, and most importantly, our system operates only inside folders the user has explicitly granted, so the blast radius of an incorrect plan is bounded by construction rather than by the agent's good behaviour.

The paper also supplies a warning we have adopted. Because output quality tracked input context so strongly, our design surfaces the files and context being used before proposing a change, rather than leaving the user to guess what the system considered.